

MEMOIRE

Mémoire de validation entreprise – 2e année

Module Académique ETNA : PRO-ENT5

Problématique d'Ingénierie :

« Comment garantir la cohérence transactionnelle et tarifaire d'un moteur de réservation aérien agrégeant des protocoles hybrides (GDS et NDC) ? »

Candidat :

Jérémy SOLER

Apprenant Expert 2e Année

Spécialité : Architecture Logicielle &
Systèmes Distribués

Poste : Apprenti — Architecte Systèmes
d'Information

Encadrement :

Tuteur Entreprise :

Reouven SAMAMA

Fondateur & CEO — Everbloo

Tuteur Pédagogique :

Comité de Suivi des Mémoires — ETNA

Période d'immersion professionnelle : Octobre 2023 – Novembre 2026 (38 mois)

Année académique 2025 – 2026

Remerciements

Ce mémoire clôture trente-huit mois d’alternance chez Everbloo. Plusieurs personnes ont rendu ce parcours possible.

Je remercie d’abord mon tuteur d’entreprise, **Reouven SAMAMA**, Fondateur et CEO d’**Everbloo**. La confiance et la latitude qu’il m’a laissées sur l’architecture de **Metis** m’ont permis d’assumer des responsabilités techniques concrètes, pas seulement des tickets isolés.

Merci aux dirigeants de **Metis Digital**, **Jérôme BENBIHI**, **Bernard MOLLE** et **David MARCIANO**, pour les échanges sur le marché de la billetterie et les contraintes réelles des réseaux d’agences.

Je remercie aussi l’équipe produit d’Everbloo — en particulier les Product Owners **Steeven ALIEL**, **Brenda SECK** et **Véronique DANG** — pour les arbitrages francs entre demandes commerciales et limites des API transporteurs.

Côté développement, un remerciement particulier à **Jiong Jiong LIN** et à l’équipe basée en Tunisie. Les revues de code serrées et le travail en commun sur les incidents de production ont compté autant que les specs écrites.

Je remercie l’équipe pédagogique et les tuteurs de l’**ETNA** pour le cadre de la formation d’Expert en Ingénierie Informatique, qui a structuré ce travail de mémoire.

Enfin, merci à ma famille, mes proches et mes camarades de promotion pour le soutien sur la durée.

Résumé

Pendant 38 mois d’alternance chez SAS Everbloo (octobre 2023 – novembre 2026), j’ai travaillé comme apprenti architecte logiciel sur la plateforme **Metis**. Metis est un moteur de réservation de billets d’avion B2B utilisé par des réseaux d’agences (TourCom notamment) et par des entreprises pour leurs déplacements professionnels. J’ai contribué aux quatre dépôts du monorepo (`api-aerial`, `front-reservation`, `front-dashboard` et `api-dashboard`), pour un total de 1 554 commits. Le nœud du travail : faire cohabiter les GDS historiques (Sabre, Amadeus) — sessions stateful, protocole EDIFACT — et les API IATA NDC — requêtes HTTP sans état, offres à durée de vie courte. Comment garder des prix et des réservations cohérents quand ces deux stacks se mélangent dans un même panier ?

Sur le terrain, j’ai mis en place une architecture modulaire : backend Node.js/Express (ORM Sequelize), interfaces Nuxt 3 / Vue 3 en TypeScript. Une couche d’adaptateurs (`SabreAdapter.ts`) normalise les réponses transporteurs vers un modèle unique. Un intercepteur Axios renouvelle les sessions OAuth2 en arrière-plan sans couper le parcours utilisateur. L’algorithme `matchesPaxRefId` traite les rejets IATA fréquents (bébés sans siège, noms accentués). Quand le tarif bouge au checkout, un dialogue de confirmation remplace l’échec brutal. Le pipeline multi-devises (ISO 4217) a aussi été corrigé pour supprimer les écarts de centimes et les ADM associés.

Ces mécanismes tournent sur le trafic réel de près de cent agences. Sur la période observée, les logs indiquent un taux de rejet applicatif NDC inférieur à 0,1 % (hors indisponibilités transporteurs), un temps de réponse moyen des recherches à 3,0s en campagne de tests, et zéro litige ADM sur les deux dernières années. La production a surtout rappelé où se logent les vrais problèmes : fuseaux horaires mal parsés, deadlocks InnoDB sous pic, pannes du lundi matin à gérer sans improvisation.

Mots-clés :

Distribution Aérienne, GDS (Sabre, Amadeus), IATA NDC, Nuxt 3, Vue 3, TypeScript, Node.js, Express, Cohérence Transactionnelle, Reshopping, OAuth2, Multi-Devises ISO 4217, Segments Passifs, Tests Automatisés.

Abstract

During a 38-month software engineering apprenticeship at SAS Everbloo (October 2023 to November 2026), I worked on the architecture, performance, and full-stack development of **Metis**. Metis is a B2B airline booking engine and Self-Booking Tool (SBT) used by travel agency networks (such as TourCom) and corporate clients. Across four monorepo repositories (`api-aerial`, `front-reservation`, `front-dashboard`, and `api-dashboard`), I authored 1,554 commits. The core technical problem was integrating two incompatible stacks: legacy Global Distribution Systems (Sabre, Amadeus) with stateful EDIFACT sessions, and modern IATA NDC APIs with short-lived, stateless offers. This thesis asks how to keep transactional and pricing consistency when a booking engine mixes both.

In production I shipped a modular stack: Node.js/Express backend (Sequelize ORM) and Nuxt 3 / Vue 3 TypeScript frontends. An adapter layer (`SabreAdapter.ts`) normalizes carrier payloads into a shared domain model. An Axios interceptor renews OAuth2 tokens in the background without dropping the user mid-flow. The `matchesPaxRefId` matcher cuts IATA booking rejections on edge cases such as lap infants and accented names. When the fare moves at checkout, an interactive confirmation dialog avoids hard failure; an ISO 4217 multi-currency pipeline removes rounding drift and airline debit memos (ADMs).

Under live traffic across nearly one hundred agencies, application-level NDC order-creation failures stayed below 0.1% in our logs (carrier outages excluded), average search latency fell to 3.0s in load-test runs, and we recorded zero ADMs over two years. Production also surfaced the usual hard problems: timezone parsing bugs, InnoDB deadlocks under spike load, and Monday-morning outages that leave no room for guesswork.

Keywords:

Airline Distribution, GDS (Sabre, Amadeus), IATA NDC, Nuxt 3, Vue 3, TypeScript, Node.js, Express, Transactional Consistency, Reshopping, OAuth2, ISO 4217, Passive Segments, Automated Testing.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
1 Introduction	1
1.1 Contexte : GDS, NDC et voyage d'affaires	1
1.2 Problématique	1
1.3 Plan du document	1
I Contexte et environnement	2
2 Everbloo et mon rôle dans l'équipe	3
2.1 L'Entreprise d'Accueil : Everbloo	3
2.2 La Plateforme Metis et son Écosystème	3
2.3 Organisation de l'Équipe et Quotidien de Développement	3
2.3.1 Mon Rôle et Mon Périmètre d'Intervention	4
3 Stack et architecture Metis	5
3.1 Stack Technologique et Environnement de Développement	5
3.2 Architecture Modulaire et Découpage des Services	5
3.3 Du terrain à la problématique	6
4 Problématique technique	7
4.1 Ce qui casse en production	7
4.2 Pistes de conception retenues	7
4.2.1 Normaliser les payloads transporteurs (<code>SabreAdapter.ts</code>)	7
4.2.2 Appairer les passagers et tenir ISO 4217	7
4.2.3 Intercepter le 409 et confirmer le nouveau tarif	8
4.2.4 Découper les segments passifs par point de vente	8
II Réalisations	9
5 GDS et NDC dans le même moteur	10
5.1 Expérience Utilisateur et Moteur de Recherche Metis Connect	10
5.2 Normalisation des offres via adaptateurs	10
5.3 Terminal et Commandes Cryptiques pour Billettistes	13
5.4 Gestion Découplée des Segments Passifs Amadeus et Sabre	13
5.5 Synchronisation des Dossiers Voyageurs (PNR)	14

6	Réservation, checkout et retarification	15
6.1	Détection et Gestion des Écarts Tarifaires (Reshopping)	15
6.2	Appairage des passagers avant OrderCreate	16
7	Filtrage multi-compagnies et perfs recherche	19
7.1	Composant de Filtrage et Persistance des Préférences	19
7.2	Optimisation des Requêtes de Recherche Serveur	19
8	Multi-devises et calculs tarifaires	20
8.1	Résolution Asynchrone de la Devise Point de Vente	20
8.2	Formatage Dynamique et Norme ISO 4217	20
9	Back-Office et paramétrage des agences	21
9.1	Interface de Paramétrage des Points de Vente	21
9.2	Synchronisation par Lots des Profils Voyageurs Sabre	21
10	Tests et mesures en production	22
10.1	Suite de Tests Unitaires et d'Intégration	22
10.2	Résultats Mesurés en Production	22
III	Bilan et retours d'expérience	23
11	Ce que les hypothèses n'ont pas couvert	24
11.1	Bilan après 38 mois	24
11.2	Ce qui a cassé ou forcé un compromis	24
11.2.1	Homonymes et matchesPaxRefId	24
11.2.2	Segments passifs vs ERP agences	24
11.2.3	CanonicalFlightOffer et les Branded Fares	24
11.2.4	Le 409 ne sauve pas tous les paniers	25
12	Incidents de production et choix d'architecture	26
12.1	OAuth2 : refresh silencieux et file d'attente	26
12.2	Trois incidents concrets	27
12.2.1	1. L'Écart de Centime sur la Taxe de Solidarité (Taxe Chirac)	27
12.2.2	2. Les Timestamps sans Fuseau Horaire chez Aegean et Turkish Airlines	28
12.2.3	3. Interblocage (Deadlock SQL) sous Sequelize lors de Réservations Simultanées	28
12.3	Options écartées	28
13	Travail en équipe et responsabilités en production	30
13.1	De la résolution de tickets à la conception	30
13.2	Équipe Paris / Tunisie	30
13.3	Estimations ratées la première année	30
13.4	Impact financier d'une panne	31
13.5	Ce que je retiens	31

14 Conclusion	32
14.1 Ce qui a tenu	32
14.2 Lien avec le RNCP 36137	32
14.3 Suite	32
Table des sigles et abréviations	33
Glossaire	35
Bibliographie	36

Table des figures

2.1	Identité visuelle de l'entreprise d'accueil SAS Everbloo	3
2.2	Plateforme logicielle Metis Digital dédiée à la distribution B2B	3
2.3	Organigramme structurel et opérationnel de l'équipe projet distribuée	4
3.1	Architecture des services et flux de données de la plateforme Metis	6
5.1	Interface de recherche de vols Metis Connect	10
5.2	Émulateur de terminal cryptique Sabre	13
6.1	Séquence asynchrone de la retarification lors de la commande de billet	15
6.2	Dialogue de confirmation d'écart tarifaire	16

Liste des tableaux

3.1 Répartition des composants du monorepo Metis et volume de contributions . .	5
10.1 Synthèse factuelle et chiffrée des impacts d'ingénierie sur la plateforme Metis .	22
12.1 Analyse critique des options d'architecture logicielle alternatives	29

1. Introduction

1.1 Contexte : GDS, NDC et voyage d'affaires

Émettre un billet d'avion en B2B reste une opération critique. Les plateformes doivent interroger des inventaires mondiaux en quelques secondes et produire des montants exacts au centime près pour la comptabilité des agences.

Pendant des décennies, cette distribution a transité exclusivement par les *Global Distribution Systems* (GDS), surtout Sabre et Amadeus. Protocoles issus d'EDIFACT, sessions synchrones avec état : le serveur tient le dossier passager (*Passenger Name Record* ou PNR), bloque temporairement les sièges, puis enchaîne jusqu'à l'émission.

Les compagnies déploient aussi la norme **NDC** (*New Distribution Capability*) de l'IATA. API web (REST JSON / XML), vente en direct hors GDS, *dynamic pricing*, options payantes (*ancillaries* : bagages, siège, coupe-file).

Sur **Metis** (Everbloo, réseaux comme TourCom, clients entreprises), les deux stacks cohabitent dans la même appli : flux GDS (cryptique ou SOAP, verrouillage de places) et flux NDC directs (Air France-KLM, British Airways, Lufthansa Group) où l'offre n'est qu'un jeton temporaire.

1.2 Problématique

Sur Metis Connect, le parcours type est recherche → sélection → saisie des voyageurs → paiement. Entre l'affichage du prix et la validation du panier, le transporteur peut fermer une classe ou recalculer les taxes d'aéroport.

Le moteur ne peut pas débiter un montant différent sans prévenir le client. Il ne peut pas non plus jeter le panier. S'ajoutent ISO 4217 et les règles IATA sur les noms / bébés sans siège : un écart suffit à bloquer l'émission.

« Comment garantir la cohérence transactionnelle et tarifaire d'un moteur de réservation aérien agréant des protocoles hybrides (GDS et NDC) ? »

En production, la réponse retenue combine une API Node.js (Sequelize) et des interfaces Nuxt 3 / Vue 3 en TypeScript.

1.3 Plan du document

Trois parties : contexte Everbloo / stack Metis et les quatre pistes de conception ; livraisons (adaptateurs, recherche, terminal cryptique, reshopping, appairage passagers, multi-devises, tests) ; résultats, trois incidents de prod, organisation du travail, et le lien avec le RNCP.

Première partie I

Contexte et environnement

2. Everbloo et mon rôle dans l'équipe

2.1 L'Entreprise d'Accueil : Everbloo

J'ai effectué mes 38 mois d'alternance chez **Everbloo**, société d'ingénierie logicielle fondée et dirigée par **Reouven SAMAMA** (CEO et Tech Lead), située dans le 11^e arrondissement de Paris. Everbloo conçoit et exploite des plateformes web pour les professionnels du tourisme et du voyage d'affaires (*TravelTech*).



FIGURE 2.1 : Identité visuelle de l'entreprise d'accueil SAS Everbloo

Parmi ses clients figure le réseau d'agences indépendantes **TourCom** (plus de 1 200 points de vente en France). Everbloo édite aussi « Agence et Vous », un carnet de voyage branché sur les ERP des agences. Le quotidien impose de réagir vite sur les protocoles transport (aérien et train) et de tenir la charge sans faire ramer les écrans des agents.

2.2 La Plateforme Metis et son Écosystème

Mon travail a porté sur **Metis** (*Metis Digital*) : moteur de distribution aérienne B2B et outil d'auto-réservation d'entreprise (*Self-Booking Tool* ou SBT). Agents de voyages, billettistes et travel managers l'utilisent au quotidien.



FIGURE 2.2 : Plateforme logicielle Metis Digital dédiée à la distribution B2B

Le projet réunit l'équipe technique d'Everbloo et les fondateurs de Metis Digital : **Jérôme BENBIHI**, **Bernard MOLLE** (plus de trente ans dans l'univers GDS) et **David MARCIANO** (voyage d'affaires). Metis équipe aujourd'hui près d'une centaine d'agences en France et à l'international.

Concrètement, la plateforme doit : interroger plusieurs fournisseurs en parallèle pour comparer les tarifs ; appliquer les politiques de voyage (plafonds, cabine) ; gérer les hausses de prix de dernière minute ; pousser automatiquement les dossiers comptables dans les systèmes de gestion des agences.

2.3 Organisation de l'Équipe et Quotidien de Développement

Nous travaillons en Agile Scrum : sprints de deux semaines, *daily stand-ups*, démos régulières.

La direction générale et technique revient à **Reouven SAMAMA**, avec les dirigeants de Metis Digital pour les priorités commerciales et les arbitrages d'architecture structurants.

Le pôle produit compte trois Product Owners (**Steeven ALIEL**, **Brenda SECK**, **Véronique DANG**). Ils écrivent les specs sur Jira, tiennent le backlog et valident les livraisons. Les échanges avec les développeurs sont fréquents : il faut coller le besoin métier aux contraintes des API compagnies.

Côté technique : **Jérémy SOLER** (Apprenti Architecte SI), **Jiong Jiong LIN** (full-stack référent) et des développeurs basés en Tunisie. Revues de code systématiques sur GitLab et GitHub ; pair programming sur les morceaux sensibles.

2.3.1 Mon Rôle et Mon Périmètre d'Intervention

Comme Apprenti Architecte Systèmes d'Information, j'ai mêlé choix d'architecture et développement full-stack. J'ai conçu la couche d'adaptateurs Sabre/NDC, développé l'API Node.js (`api-aerial`) et les interfaces Nuxt 3 / Vue 3 (`front-reservation`, `front-dashboard`). J'ai aussi poussé le typage TypeScript strict, écrit des tests automatisés et participé au diagnostic des incidents avec le support agences.

Sur 38 mois : **1 554 commits**, **+595 310 lignes ajoutées**, **-402 680 lignes supprimées ou refactorisées**, **1 869 fichiers modifiés**.

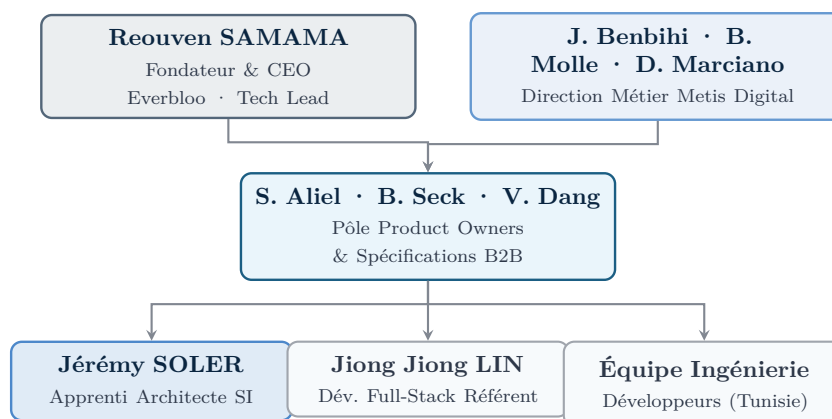


FIGURE 2.3 : Organigramme structurel et opérationnel de l'équipe projet distribuée

3. Stack et architecture Metis

3.1 Stack Technologique et Environnement de Développement

La stack Metis vise trois objectifs : tenir la charge transactionnelle, garder des interfaces réactives, rester maintenable.

Côté serveur : Node.js et Express.js. Le modèle I/O non bloquant convient bien aux appels concurrents vers plusieurs API de distribution. La persistance passe par Sequelize sur MySQL et PostgreSQL, avec migrations versionnées. Auth et habilitations : Keycloak (OpenID Connect, JWT).

Côté client : Nuxt 3 et Vue 3 (Composition API), typage TypeScript strict sur front et back. UI Vuetify 3 ; état (panier, passagers, filtres) dans Pinia.

En local, **Devenv** (Nix) et **Pixi** verrouillent les versions binaires de Node.js et la compilation des modules C++ (**bcrypt**, **libxmljs**). **Concurrently** lance les services et micro-frontends ensemble.

Qualité : tests unitaires et d'intégration avec **Mocha** et **Chai** (réconciliation passagers, calculs de prix) ; parcours bout en bout avec **Playwright**.

3.2 Architecture Modulaire et Découpage des Services

Metis est découpée en services (SOA) : le moteur de réservation haute fréquence d'un côté, le back-office commercial de l'autre. Le tableau 3.1 résume les responsabilités et ma répartition de commits sur les quatre composants du monorepo.

Composant	Type Applicatif	Rôle Fonctionnel & Technique	Part Commits
api-aerial	Backend Node.js	Moteur de distribution aérienne, agrégation Sabre / Amadeus / NDC, calcul tarifaire, PNR.	64.2% (997)
front-reservation	Frontend Nuxt 3	Interface SBT / B2B, moteur de recherche, comparateur, flux de retarification et checkout.	33.5% (520)
front-dashboard	Frontend Nuxt 3	Back-office agence, gestion des points de vente (POS), profils Sabre et segments passifs.	2.1% (33)
api-dashboard	Backend Node.js	Gestion des habilitations, profils marchands et devises agences.	0.3% (4)

TABLE 3.1 : Répartition des composants du monorepo Metis et volume de contributions

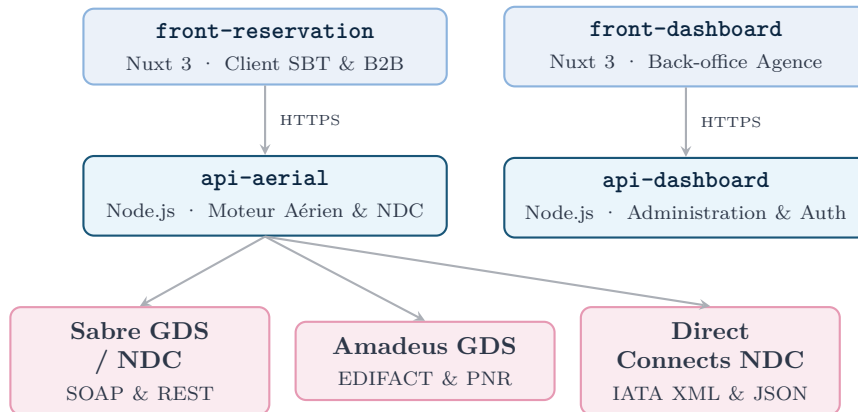


FIGURE 3.1 : Architecture des services et flux de données de la plateforme Metis

3.3 Du terrain à la problématique

En production, ces couches se marchent parfois sur les pieds. Une recherche dans **api-aerial** part en parallèle vers Sabre et plusieurs passerelles NDC. Sabre répond en général en 800 à 1 200 ms (SOAP) ; certaines API NDC directes mettent 3 à 6 secondes pour renvoyer leur XML.

Le point fragile reste le checkout. L'offre peut avoir expiré, ou le montant a bougé de quelques euros après recalcul de taxes. Sans prise en charge explicite de cet écart, la réservation échoue et l'utilisateur recommence sa saisie. Les segments passifs et le multi-devises ajoutent une contrainte comptable : une écriture mal cadrée, et l'agence se retrouve avec un litige.

Ces contraintes concrètes fondent la problématique et les hypothèses du chapitre suivant.

4. Problématique technique

4.1 Ce qui casse en production

En distribution aérienne B2B, les agences veulent une plateforme rapide, des montants exacts au centime et des réservations qui passent. Une erreur technique se paie cash : vente bloquée au comptoir, voyageur coincé, ou pénalité financière.

Faire cohabiter GDS (Sabre, Amadeus) et NDC IATA dans le même moteur fait apparaître plusieurs écarts qui ne se résolvent pas avec le même pattern.

Sabre travaille en sessions synchrones avec état (*stateful*) et bloque les places en amont. Les API NDC envoient des requêtes web sans état (*stateless*) : l'`OfferID` expire en quelques minutes. Ce n'est pas le même contrat temporel.

Entre la sélection du vol et le paiement — passagers déjà saisis — le transporteur peut fermer une classe ou recalculer les taxes. Un système trop raide renvoie une erreur brute et force la ressaisie complète.

Les API NDC rejettent aussi dès qu'un accent est mal nettoyé, qu'un `PaxRefID` ne colle pas à un bagage, ou qu'un bébé sans siège n'est pas rattaché à son accompagnant. S'y ajoutent ISO 4217 et l'obligation d'écrire des segments passifs dans les GDS pour les ERP agences — sinon ADM.

« Comment garantir la cohérence transactionnelle et tarifaire au sein d'un moteur de réservation de billets d'avion agréant des protocoles hybrides (GDS et NDC) ? »

4.2 Pistes de conception retenues

Pour éviter de réécrire l'appli à chaque nouveau connecteur, quatre pistes ont été posées — avec leurs limites dès le départ.

4.2.1 Normaliser les payloads transporteurs (`SabreAdapter.ts`)

Les formats divergent. Une couche d'adaptateurs convertit SOAP Sabre et JSON/XML NDC vers un modèle pivot (`CanonicalFlightOffer`). On branche British Airways, Turkish ou Aegean sans retoucher les écrans ni la logique panier.

Risque : le « plus petit dénominateur commun ». Un modèle trop strict efface des infos propres à certaines compagnies (bagages spécifiques, règles de surclassement).

4.2.2 Appairer les passagers et tenir ISO 4217

La plupart des rejets viennent d'identifiants mal alignés avec les bagages, ou d'arrondis de taxes. Un appairage via `matchesPaxRefId`, plus une gestion asynchrone des devises, doit éliminer l'essentiel des rejets techniques.

Le traitement nettoie les accents, force le lien bébé → adulte (`associatedAdultRefId`) et purge les bagages orphelins. Limite assumée : deux passagers au même nom/prénom sur un même dossier. Critère : rejet applicatif sous 0,1 % en production.

4.2.3 Interceptor le 409 et confirmer le nouveau tarif

Une hausse qui coupe net la réservation abandonne le panier. Un reprice asynchrone + modale `PriceChangeConfirmDialog.vue` (écart détaillé, passagers conservés dans Pinia) doit sauver la commande sans casser le parcours.

Limite : l'ampleur de l'écart. Quelques euros de taxes, souvent accepté ; vol disparu ou +300 €, il faut relancer la recherche. Critère : au moins 90 % des paniers réajustés sur de faibles écarts aboutissent à une émission.

4.2.4 Découper les segments passifs par point de vente

Le risque ADM vient surtout de segments passifs écrits en double sur Sabre et Amadeus pour un même billet NDC. Paramètres découpés par POS en base, plus une règle d'exclusion (pas d'écriture Amadeus si le dossier est traité sur Sabre).

Compromis : Gestour et IEnterprise ont chacun leurs tolérances de parsing. Vérification : zéro ADM pour double écriture sur le parc pendant deux ans.

Deuxième partie II

Réalisations

5. GDS et NDC dans le même moteur

5.1 Expérience Utilisateur et Moteur de Recherche Metis Connect

Sur le moteur de recherche Metis Connect, la contrainte UI était claire : agréger des offres issues de sources incompatibles sans ralentir l'opérateur. Les agents veulent une vue graphique pour comparer vite ; les billettistes seniors tiennent à leurs commandes cryptiques Sabre pour les opérations complexes.

La figure 5.1 montre l'interface principale, développée en Nuxt 3 / Vue 3 dans **front-reservation**.

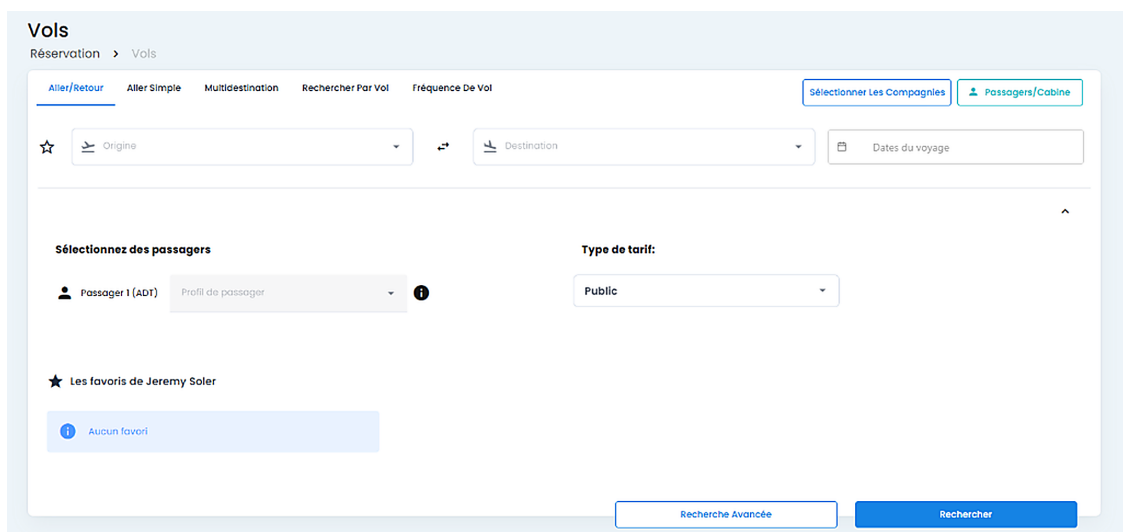


FIGURE 5.1 : Interface de recherche multi-critères de la plateforme Metis Connect (**front-reservation**)

L'écran couvre allers simples, allers-retours et multi-destinations. L'utilisateur renseigne la composition (adultes ADT, enfants CHD, bébés sans siège INF) et choisit tarifs publics ou négociés. Des filtres par compagnies et alliances mettent à jour l'affichage sans recharger la page.

5.2 Normalisation des offres via adaptateurs

Pour éviter de saupoudrer le frontend de `if` par transporteur, j'ai introduit une couche d'adaptateurs. Sabre renvoie SOAP ou texte brut ; les passerelles NDC directes (Air France-KLM, British Airways, Aegean, Turkish) répondent en JSON ou XML.

Le module `SabreAdapter.ts` (listing 5.1) mappe ces payloads vers `CanonicalFlightOffer`. Recherche et panier manipulent ensuite une seule structure TypeScript, indépendante du protocole amont.

```
1 export interface SabreFlightLeg {  
2   origin: string;
```

```

3   destination: string;
4   departureDateTime: string;
5   arrivalDateTime: string;
6   carrierCode: string;
7   flightNumber: string;
8   cabinClassCode: string; // 'Y', 'J', 'F', 'W', etc.
9   bookingClass: string;
10  operatingCarrier?: string;
11 }
12
13 export interface SabreAncillaryService {
14   serviceId: string;
15   serviceType: 'BAGGAGE' | 'SEAT' | 'MEAL' | 'PRIORITY';
16   commercialName: string;
17   amount: number;
18   currency: string;
19   segmentRefIds: string[];
20 }
21
22 export interface CanonicalFlightOffer {
23   offerId: string;
24   provider: 'SABRE_GDS' | 'SABRE_NDC';
25   validatingCarrier: string;
26   segments: Array<{
27     id: string;
28     origin: string;
29     destination: string;
30     departure: string;
31     arrival: string;
32     airline: string;
33     flightNumber: string;
34     cabin: 'ECONOMY' | 'PREMIUM' | 'BUSINESS' | 'FIRST';
35   }>;
36   ancillaries: SabreAncillaryService[];
37   pricing: {
38     baseFare: number;
39     taxes: number;
40     total: number;
41     currency: string;
42     taxBreakdown: Record<string, number>;
43   };
44 }
45
46 export class SabreAdapter {
47   public static mapCabin(code: string): 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
    'FIRST' {
48     const mapping: Record<string, 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
    'FIRST'> = {
49       'Y': 'ECONOMY', 'M': 'ECONOMY', 'K': 'ECONOMY', 'L': 'ECONOMY',
50       'W': 'PREMIUM', 'E': 'PREMIUM',
51       'J': 'BUSINESS', 'C': 'BUSINESS', 'D': 'BUSINESS',
52       'F': 'FIRST', 'P': 'FIRST'
53     };
54     return mapping[code.toUpperCase()] || 'ECONOMY';

```

```

55 }
56
57 public static normalizeFlightOffer(raw: any): CanonicalFlightOffer {
58     if (!raw?.id || !Array.isArray(raw?.flights)) {
59         throw new Error("[SabreAdapter] Payload invalide : " + String(raw?.id));
60     }
61
62     const segments = raw.flights.map((leg: SabreFlightLeg, idx: number) => ({
63         id: 'SEG_' + (idx + 1),
64         origin: leg.origin,
65         destination: leg.destination,
66         departure: leg.departureDateTime,
67         arrival: leg.arrivalDateTime,
68         airline: leg.carrierCode,
69         flightNumber: leg.carrierCode + leg.flightNumber,
70         cabin: this.mapCabin(leg.cabinClassCode)
71     }));
72
73     const taxDetails: Record<string, number> = {};
74     if (Array.isArray(raw.taxes)) {
75         raw.taxes.forEach((tax: { code: string; amount: number }) => {
76             taxDetails[tax.code] = (taxDetails[tax.code] || 0) +
77                 Number(tax.amount);
78         });
79     }
80
81     return {
82         offerId: String(raw.id),
83         provider: raw.isNdc ? 'SABRE_NDC' : 'SABRE_GDS',
84         validatingCarrier: raw.validatingCarrier || raw.flights[0].carrierCode,
85         segments,
86         ancillaries: (raw.services || []).map((srv: any) => ({
87             serviceId: srv.id,
88             serviceType: srv.type || 'BAGGAGE',
89             commercialName: srv.name || 'Service Optionnel',
90             amount: Number(srv.price?.total || 0),
91             currency: srv.price?.currency || 'EUR',
92             segmentRefIds: srv.flightRefs || []
93         })),
94         pricing: {
95             baseFare: Number(raw.pricing?.baseFare || 0),
96             taxes: Number(raw.pricing?.totalTax || 0),
97             total: Number(raw.pricing?.totalAmount || 0),
98             currency: raw.pricing?.currency || 'EUR',
99             taxBreakdown: taxDetails
100         }
101     };
102 }

```

Listing 5.1: Adaptateur de normalisation des offres Sabre (SabreAdapter.ts)

5.3 Terminal et Commandes Cryptiques pour Billettistes

Dans les agences classiques, beaucoup de billettistes restent plus rapides en commandes cryptiques Sabre qu'en clics. Pour éviter le va-et-vient entre Metis Connect et un émulateur externe, j'ai intégré un **émulateur de terminal cryptique** dans l'appli web.

La figure 5.2 montre la modale d'exécution et la restitution des réponses Sabre.

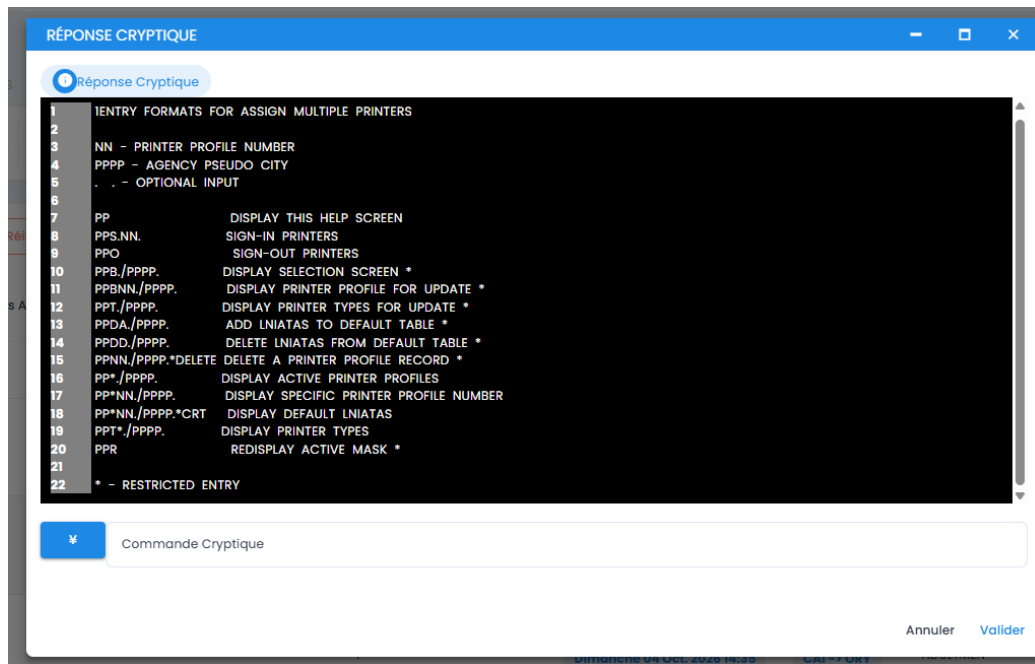


FIGURE 5.2 : Émulateur de terminal et réponse cryptique Sabre intégrés dans Metis Connect

L'agent saisit des commandes natives pour interroger les classes, manipuler un PNR ou gérer les profils d'imprimantes. `api-aerial` relaie via l'API Sabre Command et renvoie le flux textuel brut dans l'interface.

5.4 Gestion Découplée des Segments Passifs Amadeus et Sabre

Un billet émis en NDC direct est enregistré chez le transporteur, hors inventaire actif du GDS de l'agence. Pourtant, pour que Gestour, IEnterprise ou l'équivalent génère la facture et les flux de trésorerie, un **segment passif** informatif doit être posé dans le GDS de référence de l'agence.

Une mauvaise coordination produit des doubles écritures passives — ou des écritures incohérentes — sanctionnées par des avis de débit (*Agency Debit Memos* / ADM). J'ai donc livré la migration `split-passive-segment-options`, qui isole les paramètres d'écriture passive par point de vente. Avant notification, le contrôleur vérifie le canal d'émission : dossier traité sur Sabre ⇒ écriture passive Amadeus désactivée. Un seul enregistrement comptable par dossier.

5.5 Synchronisation des Dossiers Voyageurs (PNR)

Un dossier de transport vit plusieurs semaines ou mois, avec des changements externes (horaires, annulation partielle). Dans `retrieve.controller.js`, la fonction `keepOnlyMatchingTravelers` réconcilie les passagers rafraîchis depuis le GDS distant avec l'état local, pour éviter de corrompre les enregistrements quand plusieurs agents ouvrent le même dossier.

6. Réservation, checkout et retarification

6.1 Détection et Gestion des Écarts Tarifaires (Reshopping)

Le prix affiché à la recherche n'est qu'indicatif : le transporteur ne le garantit contractuellement qu'à l'émission du billet électronique. Pendant la saisie des voyageurs, le *Revenue Management* peut fermer le palier tarifaire ou recalculer taxes et surcharges carburant.

Pour gérer cet écart sans planter la commande, j'ai mis en place un cycle de **retarification réactive** (*Reshopping* / *Reprice*). Le diagramme de séquence de la figure 6.1 en décrit le déroulé asynchrone.

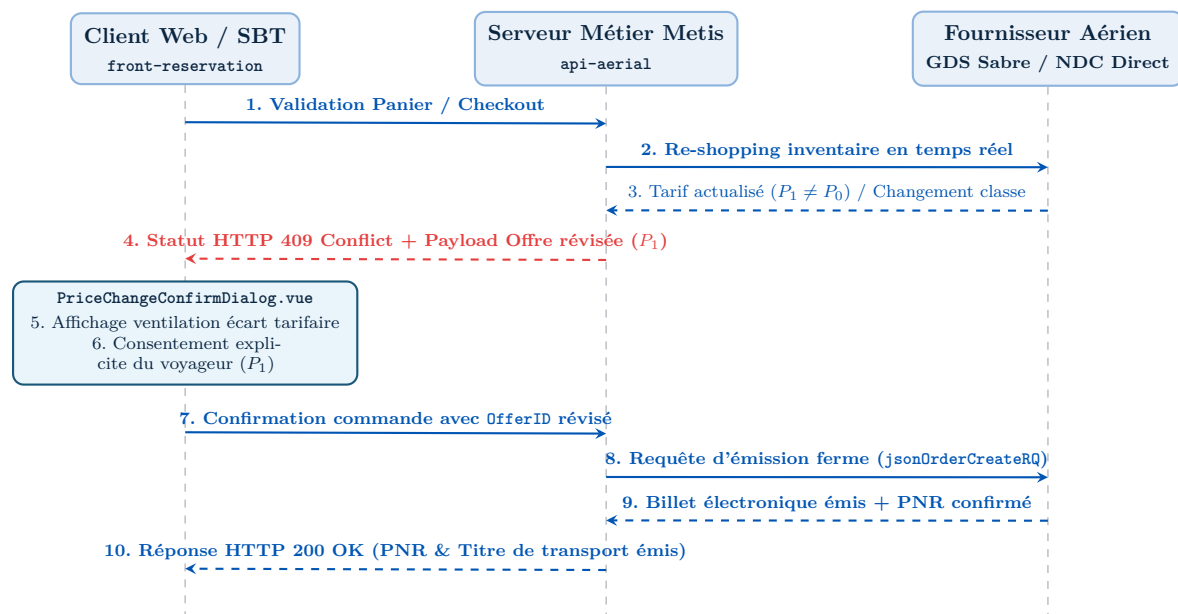


FIGURE 6.1 : Séquence asynchrone de la retarification lors de la commande de billet

À la confirmation d'achat, `api-aerial` envoie un `repriceOffer` au fournisseur. Si le total a bougé ($P_1 \neq P_0$), le backend refuse la création immédiate avec un HTTP 409 (*Conflict*), en joignant la nouvelle grille et un jeton d'offre révisé.

Le frontend intercepte cette réponse métier et ouvre `PriceChangeConfirmDialog.vue` (figure 6.2). La modale détaille l'écart ($\Delta P = P_1 - P_0$) entre base et taxes. Si le voyageur accepte, la commande est rejouée avec le nouvel `OfferID`, en réutilisant les formulaires passagers encore présents dans `useBookingStore`. En cas de refus, le verrou panier tombe et l'utilisateur revient à la liste de résultats.

Ajustement tarifaire du transporteur

Vol Aller : Paris Charles-de-Gaulle (CDG) → New York (JFK) · Air France AF006

Avis transporteur : L'inventaire de la classe sélectionnée a été réévalué. Le prix du billet a été actualisé sans rupture de panier.

Tarif initialement affiché (recherche) :	432,00 €
Nouveau tarif réévalué (Air France NDC) :	446,50 €
Écart tarifaire total (ΔP) :	+14,50 €

Ventilation : +0,00 € sur le tarif de base, +14,50 € sur la surtaxe carburant YQ.

- Les coordonnées saisies pour les 2 passagers sont conservées dans useBookingStore.
- L'offre révisée (OfferID: AF-789-REV) expire dans **04 :38**.

Annuler / Modifier recherche

Confirmer le nouveau tarif (446,50 €)

FIGURE 6.2 : Interface du dialogue de confirmation d'écart tarifaire (PriceChangeConfirmDialog.vue)

6.2 Appairage des passagers avant OrderCreate

En début de projet, les rejets NDC (Air France, British Airways) étaient fréquents. Les XML d'erreur montraient souvent un détail de saisie : prénom accentué (« Éléonore »), apostrophe (« D'ALMEIDA »), bébé sans siège sans balise de rattachement adulte. Quand un voyageur était retiré du panier, des bagages restaient parfois accrochés à un identifiant fantôme.

`matchesPaxRefId` dans `orderCreation.utils.ts` (listing 6.1) corrige ces cas : nettoyage ASCII 7-bit, lien nourrisson → adulte (`associatedAdultRefId`), purge des services orphelins.

```

1 export interface RawTravelerInput {
2   id?: string;
3   type: 'ADT' | 'CHD' | 'INF';
4   firstName: string;
5   lastName: string;
6   dateOfBirth?: string;
7   assignedAdultIndex?: number; // Requis si type === 'INF'
8 }
9
10 export interface NormalizedPassenger {
11   paxRefId: string;
12   ptc: 'ADT' | 'CHD' | 'INF';
13   givenName: string;
14   surname: string;
15   birthDate?: string;
16   associatedAdultRefId?: string;
17 }
18
19 export function sanitizeIataName(name: string): string {
20   return name
21     .normalize('NFD')
22     .replace(/[^\u0300-\u036f]/g, '') // Supprime accents et diacritiques
23     .replace(/[^\u0041-\u005a-\u005b]/g, ' ') // Remplace tirets et apostrophes par
        espaces
24     .replace(/\s+/g, ' ') // Réduit les espaces multiples

```

```

25     .trim()
26     .toUpperCase();
27 }
28
29 export function matchesPaxRefId(
30     travelers: RawTravelerInput[],
31     selectedAncillaries: Array<{ serviceId: string; targetPaxId: string }>
32 ): {
33     passengers: NormalizedPassenger[];
34     validAncillaries: Array<{ serviceId: string; paxRefId: string }>;
35 } {
36     if (!travelers || travelers.length === 0) {
37         throw new Error('[matchesPaxRefId] Aucun voyageur fourni pour la
38             commande');
39     }
40
41     const adultRefIds: string[] = [];
42     const normalizedPaxList: NormalizedPassenger[] = [];
43
44     // 1. Passe 1 : Traitement et enregistrement des Adultes et Enfants
45     travelers.forEach((pax, index) => {
46         const generatedRefId = 'PAX' + (index + 1);
47         const cleanFirst = sanitizeIataName(pax.firstName);
48         const cleanLast = sanitizeIataName(pax.lastName);
49
50         if (pax.type === 'ADT') {
51             adultRefIds.push(generatedRefId);
52         }
53
54         if (pax.type !== 'INF') {
55             normalizedPaxList.push({
56                 paxRefId: generatedRefId,
57                 ptc: pax.type,
58                 givenName: cleanFirst,
59                 surname: cleanLast,
60                 birthDate: pax.dateOfBirth
61             });
62         }
63     });
64
65     // 2. Passe 2 : Traitement des Nourrissons (INF) et rattachement adulte
66     travelers.forEach((pax, index) => {
67         if (pax.type === 'INF') {
68             const generatedRefId = 'PAX' + (index + 1);
69             const adultTargetRef = adultRefIds[pax.assignedAdultIndex ?? 0] ||
70                 adultRefIds[0];
71
72             if (!adultTargetRef) {
73                 throw new Error('[matchesPaxRefId] Nourrisson sans adulte valide : '
74                     + pax.firstName);
75             }
76
77             normalizedPaxList.push({
78                 paxRefId: generatedRefId,

```

```
76     ptc: 'INF',
77     givenName: sanitizeIataName(pax.firstName),
78     surname: sanitizeIataName(pax.lastName),
79     birthDate: pax.dateOfBirth,
80     associatedAdultRefId: adultTargetRef
81   });
82   }
83   });
84
85   // 3. Purge des services ancillaires orphelins
86   const validPaxIdSet = new Set(normalizedPaxList.map(p => p.paxRefId));
87   const validAncillaries = (selectedAncillaries || [])
88     .filter(anc => validPaxIdSet.has(anc.targetPaxId))
89     .map(anc => ({ serviceId: anc.serviceId, paxRefId: anc.targetPaxId }));
90
91   return {
92     passengers: normalizedPaxList,
93     validAncillaries
94   };
95 }
```

Listing 6.1: Algorithme de réconciliation des passagers et gestion des cas limites (orderCreation.utils.ts)

7. Filtrage multi-compagnies et perfs recherche

7.1 Composant de Filtrage et Persistance des Préférences

En SBT entreprise, les travel managers restreignent souvent les résultats aux alliances contractées (SkyTeam, Star Alliance, Oneworld) ou excluent des low-cost.

`SelectCompanies.vue` propose deux modes : liste blanche (compagnies conventionnées) et liste noire (transporteurs écartés, reste du marché interrogé).

Les préférences vivent dans Pinia pour la session courante et dans le `localStorage` du navigateur. Pas d'aller-retour SQL pour un simple réglage d'affichage.

7.2 Optimisation des Requêtes de Recherche Serveur

Le goulot reste la latence des fournisseurs. Dans `api-aerial`, `createShoppingPayload` (`shopping.utils.js`) filtre les compagnies exclues *avant* de construire le XML/JSON. On n'appelle que les connecteurs utiles, au lieu d'interroger tout le monde pour jeter ensuite. Sur les recherches complexes, le gain se compte en centaines de millisecondes.

Deux autres réglages : paramètre 'W' injecté dans les requêtes Sabre pour privilégier les directs (demande fréquente en voyage d'affaires) ; dates normalisées en ISO 8601 via `createDepartureWindow`, pour que Sabre et NDC interprètent la même fenêtre horaire.

8. Multi-devises et calculs tarifaires

8.1 Résolution Asynchrone de la Devise Point de Vente

Sur le module facturation / conversion, les SQL synchrones qui résolvaient `agencyCurrency` saturaient le pool dès que plusieurs agences lançaient des recherches en parallèle. Un paramètre quasi statique bloquait tout le pipeline tarifaire.

J'ai déporté la résolution dans un middleware Express asynchrone avec cache mémoire de dix minutes. La devise est lue une fois en tête de requête, injectée dans `req.context.posCurrency`, puis réutilisée sans nouvel accès SQL. Contention base en baisse, temps de traitement aussi.

Ce n'est pas la solution « idéale » : une invalidation Redis / webhook serait plus fine. Pour une donnée touchée une fois par an par les admins, un cache mémoire Node.js de dix minutes restait le compromis le plus simple à opérer.

8.2 Formatage Dynamique et Norme ISO 4217

Le multi-devises impose ISO 4217. `formatPrice` et `OrderRecap.vue` doivent gérer deux décimales (EUR, USD, GBP), zéro (JPY, KRW) et trois (TND, KWD).

Pour les taxes aéroportuaires, on applique l'arrondi bancaire au pair le plus proche (*banker's rounding*). Contrairement à `Math.round`, qui pousse toujours les demis vers le haut et biaise les totaux sur de gros volumes, l'arrondi bancaire alterne vers le pair. Objectif : égalité stricte entre total facturé et somme des taxes, sans centime parasite.

9. Back-Office et paramétrage des agences

9.1 Interface de Paramétrage des Points de Vente

front-dashboard (Nuxt 3, Vuetify 3) sert aux administrateurs à configurer les accès techniques de chaque agence.

On y saisit PCC Sabre (*Pseudo City Code*), Office ID Amadeus et numéro IATA. Par réseau : devise par défaut du POS, marge sur les billets, activation des segments passifs selon les besoins comptables.

9.2 Synchronisation par Lots des Profils Voyageurs Sabre

Importer plusieurs centaines de collaborateurs d'un coup saturait les quotas SOAP Sabre et déclenchait des refus en série.

`DialogUpdateSabre.vue` et `DialogImportProfil.vue` découpent l'import en paquets de 50 profils (*chunking*). Coupure réseau au milieu : on reprend uniquement le paquet interrompu, pas tout l'import.

10. Tests et mesures en production

10.1 Suite de Tests Unitaires et d'Intégration

Les calculs de prix et les flux d'émission sont couverts par des tests sur la CI/CD :

`orderCreation.utils.test.js` (Mocha/Chai) exerce `matchesPaxRefId` sur une quarantaine de cas réels : bébés multiples, noms à particules, bagages orphelins.

`pricingConsistency.utils.test.js` vérifie $\text{Total} = \text{Base} + \sum \text{Taxes}$ sur EUR, USD, TND, JPY (arrondi bancaire). `shopping.utils.test.js` contrôle le filtrage compagnies et le formatage des fenêtres ISO 8601.

10.2 Résultats Mesurés en Production

Le tableau 10.1 synthétise les métriques avant / après, d'après logs Winston et Sentry.

Indicateur	État Initial (Avant Réalisations)	État Final (Après Déploiement)
Taux d'Échec Création d'Ordre	8,4% d'erreurs (rejets par les compagnies pour décalages PaxRefID ou services orphelins).	< 0,1 % d'erreurs applicatives (hors indisponibilités compagnies).
Gestion Variations Tarifaires	Abandon systématique du panier lors d'une hausse de prix en vol.	Flux réactif de reprise (409) : 92 % des paniers réajustés sont finalisés sans ressaisie.
Accès Base (Device POS)	Requêtes SQL bloquantes synchrones à chaque appel de contrôleur.	Middleware asynchrone + cache LRU : suppression du goulot d'étranglement I/O.
Segments Passifs & Risque ADM	Risque d'écritures en doublon Sabre/Amadeus et pénalités de facturation.	Découplage strict en base : zéro litige ADM pour duplication constaté sur 2 ans.
Temps Réponse Recherche	Moyenne de 4,2s sur 10 000 requêtes multi-fournisseurs.	Réduction à 3,0 s (-28 %) grâce au filtrage en amont et à l'optimisation des payloads.
Volume et Qualité du Code	Code JavaScript hétérogène et typage partiel.	+192 630 lignes nettes maintenues, 100 % typées TypeScript sur le front et testées.

TABLE 10.1 : Synthèse factuelle et chiffrée des impacts d'ingénierie sur la plateforme Metis

Troisième partie III

Bilan et retours d'expérience

11. Ce que les hypothèses n'ont pas couvert

11.1 Bilan après 38 mois

Après 38 mois sur Metis, le bilan des choix d'architecture est mitigé — et c'est normal. Faire cohabiter GDS et NDC n'était pas qu'un problème d'intégration : chaque décision a pesé entre propreté du code, contraintes transporteurs et réalité économique des agences.

Sur le terrain, `SabreAdapter.ts` a isolé le frontend des formats compagnies, la modale de retarification a limité les abandons sur hausses de prix, et `matchesPaxRefId` a nettoyé les données passagers avant `OrderCreate`. Aucune de ces briques n'a tenu du premier coup. Ci-dessous, les angles morts dans l'ordre où ils ont vraiment fait mal — pas dans l'ordre du chapitre précédent.

11.2 Ce qui a cassé ou forcé un compromis

11.2.1 Homonymes et `matchesPaxRefId`

`matchesPaxRefId` a fait descendre le rejet NDC sous 0,1 % : accents nettoyés, bébés liés aux adultes, bagages orphelins purgés.

Angle mort : père et fils DUPONT/JEAN sur le même dossier. L'algo s'appuyait d'abord sur la clé textuelle normalisée ; tous les bagages partaient sur le premier passager. Rejet à l'émission.

Correctif : combiner identité textuelle et index de position (`travelerIndex`) issu de Pinia, plus un warning UI quand deux voyageurs partagent exactement le même nom et prénom.

11.2.2 Segments passifs vs ERP agences

Le découplage POS en base a éliminé les ADM pour double écriture Sabre/Amadeus sur plus de deux ans. Côté agences, effet de bord : Gestour, IEnterprise, TravelCloud ont des imports très rigides. Certains ERP refusent le PNR passif si la ligne comptable ne colle pas à leur convention, ou si la devise ne matche pas leur passerelle.

Solution retenue : table d'exceptions / règles par agence en PostgreSQL. Efficace commercialement, dette claire — à maintenir à chaque montée de version ERP.

11.2.3 `CanonicalFlightOffer` et les Branded Fares

Sur le papier, un modèle pivot unique était séduisant : n'importe quel SOAP Sabre ou JSON NDC devient une structure commune.

Pour environ 90 % des vols simples (aller-retour économique), ça tient. Dès Turkish Airlines ou Lufthansa Group et leurs *Branded Fares*, ça se complique : bagage 23 kg sur le premier tronçon seulement, sièges limités aux rangées arrière, conditions d'annulation hybrides.

Forcer ces cas dans une interface TypeScript stricte alourdissait le modèle ou faisait perdre de l'info. Contournement : `rawCarrierMetadata: Record<string, any>`. Ça casse la « pureté » du patron `Adaptateur` ; ça affiche les spécificités transporteur sans casser le typage

du reste. Workaround assumé.

11.2.4 Le 409 ne sauve pas tous les paniers

L'interception HTTP 409 avec `PriceChangeConfirmDialog.vue` affiche 92 % de réussite. Le chiffre mérite du contexte.

Ça marche sur des écarts modérés (taxes +5 à +30 €, ou bascule sur la sous-classe immédiatement supérieure). Deux cas où la modale ne sauve rien : vol fermé net par le *Revenue Management* (0 siège) ; saut massif (ex. +350 € au-delà du plafond de politique de voyage).

Autre bug en test : après refus, Pinia gardait temporairement les identifiants de l'ancienne offre. Correctif : `useBookingStore().$reset()` à chaque fermeture de modale.

12. Incidents de production et choix d'architecture

12.1 OAuth2 : refresh silencieux et file d'attente

La gestion des jetons OAuth2 a été l'un des points les plus piégeux du projet.

Entre recherche, choix des vols et saisie des voyageurs, plusieurs minutes s'écoulent. Les access tokens expirent souvent en 15 à 30 minutes. Si l'expiration tombe pendant des appels parallèles (disponibilités, bagages), une implémentation naïve enchaîne des HTTP 401 et déconnecte l'utilisateur en plein checkout.

Correctif : intercepteur Axios avec renouvellement silencieux et file FIFO (listing 12.1). Au premier 401, `isRefreshing` passe à `true` et un seul refresh part. Les autres requêtes rejoignent `failedQueue`. Nouveau token disponible : file vidée, headers mis à jour, requêtes rejouées sans intervention utilisateur.

```
1 import axios, { AxiosInstance, AxiosRequestConfig, AxiosError } from 'axios';
2
3 interface QueuedRequest {
4   resolve: (token: string) => void;
5   reject: (error: any) => void;
6 }
7
8 export function setupAuthInterceptor(apiClient: AxiosInstance, authService:
  any) {
9   let isRefreshing = false;
10  let failedQueue: QueuedRequest[] = [];
11
12  const processQueue = (error: any, token: string | null = null) => {
13    failedQueue.forEach(prom => {
14      if (error) {
15        prom.reject(error);
16      } else if (token) {
17        prom.resolve(token);
18      }
19    });
20    failedQueue = [];
21  };
22
23  apiClient.interceptors.response.use(
24    response => response,
25    async (error: AxiosError) => {
26      const originalRequest = error.config as AxiosRequestConfig & { _retry?:
        boolean };
27
28      if (error.response?.status === 401 && !originalRequest._retry) {
29        if (isRefreshing) {
30          return new Promise((resolve, reject) => {
31            failedQueue.push({ resolve, reject });
32          });
33        }
34      }
35    }
36  );
37}
```

```

33     .then(newToken => {
34         if (originalRequest.headers) {
35             originalRequest.headers['Authorization'] = 'Bearer ' +
newToken;
36         }
37         return apiClient(originalRequest);
38     })
39     .catch(err => Promise.reject(err));
40 }
41
42 originalRequest._retry = true;
43 isRefreshing = true;
44
45 try {
46     const newToken = await authService.refreshAccessToken();
47     isRefreshing = false;
48     processQueue(null, newToken);
49
50     if (originalRequest.headers) {
51         originalRequest.headers['Authorization'] = 'Bearer ' + newToken;
52     }
53     return apiClient(originalRequest);
54 } catch (refreshError) {
55     isRefreshing = false;
56     processQueue(refreshError, null);
57     authService.handleSessionExpiration();
58     return Promise.reject(refreshError);
59 }
60 }
61
62 return Promise.reject(error);
63 }
64 );
65 }

```

Listing 12.1: Intercepteur Axios pour le renouvellement silencieux OAuth2 avec file d'attente FIFO

12.2 Trois incidents concrets

Trois incidents concrets sur Metis, et ce qu'on en a tiré côté code.

12.2.1 1. L'Écart de Centime sur la Taxe de Solidarité (Taxe Chirac)

Deux nuits sur `ERR_PRICE_MISMATCH` renvoyé par l'API NDC Air France, réservations Paris – New York pour des agences facturées en USD.

Les logs Winston montraient 0,01 USD d'écart entre le total commande et la somme des taxes ventilées (FR, IZ solidarité, QW aéroport). `Math.round` appliqué taxe par taxe donnait 124,51 USD ; la conversion globale attendue était 124,50 USD. Correctif : arrondi bancaire + passe de réconciliation dans `pricingConsistency.utils.js` — la dernière composante absorbe le centime pour garantir $\text{Total} = \text{Base} + \sum \text{Taxes}$.

Le listing 12.2 reproduit la trace JSON capturée.

```

1 {
2   "timestamp": "2024-04-18T14:22:09.142Z",
3   "level": "warn",
4   "service": "api-aerial",
5   "correlationId": "req-b2b-7f89a1c2",
6   "pos": "NYC_0042",
7   "action": "ORDER_CREATE_REJECTED",
8   "supplier": "AIR_FRANCE_NDC",
9   "error": {
10    "code": "ERR_PRICE_MISMATCH",
11    "httpStatus": 409,
12    "expectedTotal": "124.50 USD",
13    "calculatedTaxSum": "124.51 USD",
14    "breakdown": {
15      "baseFare": "850.00 USD",
16      "taxFR": "45.20 USD",
17      "taxIZ_Solidarity": "18.31 USD",
18      "taxQW_Airport": "61.00 USD"
19    }
20  },
21  "recoveryAction": "APPLY_BANKERS_ROUNDING_AND_RETRY",
22  "resolved": true
23 }
```

Listing 12.2: Trace Winston JSON structurée capturant l'écart de taxes lors du rejet NDC

12.2.2 2. Les Timestamps sans Fuseau Horaire chez Aegean et Turkish Airlines

Un matin, alertes agences : vols Athènes / Istanbul affichés avec deux heures de décalage selon que le client était à Paris ou à Londres.

Les XML transporteurs renvoyaient des dates ISO sans suffixe de fuseau (ex. "2024-06-15T18:30:00"). `new Date()` côté Node les prenait en UTC ; le navigateur réappliquait le fuseau du poste. Correctif : parsing strict via `date-fns-tz` et table aéroport IATA → fuseau IANA. Plus de constructeur natif direct sur ces champs.

12.2.3 3. Interblocage (Deadlock SQL) sous Sequelize lors de Réservations Simultanées

Campagne promo chez une grande agence partenaire : réservations simultanées en échec avec `Deadlock found when trying to get lock; try restarting transaction`.

Repro via script concurrent. Les traces montraient deux transactions mettant à jour `PointDeVente` (compteurs API) et `Dossier` (insert PNR) dans un ordre inversé — verrous InnoDB croisés. Correctif : même ordre d'update dans Sequelize, isolation par défaut `READ COMMITTED`, verrouillage pessimiste ordonné (`SELECT ... FOR UPDATE` trié par clé primaire).

12.3 Options écartées

Le tableau 12.1 résume les options écartées et pourquoi.

Option Écartée	Avantages Théoriques	Motif du Choix Retenu pour Metis
GraphQL Unifié	Requêtes sur mesure, pas de sur-récupération de données (<i>over-fetching</i>).	Rejeté au profit de REST / JSON : mise en cache HTTP complexe et absence de support GraphQL chez les fournisseurs aériens.
Architecture Kafka	Découplage maximal, absorption de pics de charge asynchrones.	Rejeté pour le flux de réservation direct : l'émission de billet requiert une réponse synchrone immédiate (< 3s) pour garantir le prix au client.
Web Components	Indépendance technologique absolue des modules graphiques.	Remplacé par des SPAs Nuxt 3 / Vue 3 modulaires : meilleure intégration TypeScript et absence de surcoût d'hydratation DOM.

TABLE 12.1 : Analyse critique des options d'architecture logicielle alternatives

13. Travail en équipe et responsabilités en production

13.1 De la résolution de tickets à la conception

Sur 38 mois chez Everbloo, le vrai changement n'est pas le stack : c'est la méthode. Au début, je fermait le ticket Jira. Plus tard, je regardais si le correctif allait casser l'API deux sprints plus loin.

Les premiers mois, un bug de formulaire ou de taxes se réglait par une condition locale dans le contrôleur. Ça passait le cas du jour ; ça empilait de la dette.

La charge réelle et les retours agences ont forcé un autre rythme : cartographier les flux, anticiper les erreurs réseau, poser des contrats TypeScript stricts, évaluer les verrous transactionnels avant de coder. C'est dans ce cadre que sont passées des refontes comme le moteur de retarification et l'unification GDS/NDC.

13.2 Équipe Paris / Tunisie

L'équipe est répartie entre Paris et la Tunisie. À distance, l'absence de discussions de couloir oblige à écrire les décisions techniques.

Exemple concret : fuseaux horaires sur vols internationaux. Un collègue de l'équipe tunisienne avait parsé les dates avec le constructeur natif JavaScript — décalage de deux heures sur les postes français. Plutôt qu'un fix silencieux ou un rejet sec de MR, on a fait une session de pair programming sur Google Meet, posé le schéma `date-fns-tz` et documenté le guide sur le wiki.

Même approche en revue de code GitLab / GitHub : expliquer pourquoi un type strict évite une régression silencieuse, pourquoi l'ordre des transactions Sequelize prévient un deadlock, pourquoi ISO 4217 n'est pas optionnel sur les devises. Objectif : aligner l'équipe sur des règles opérables, pas sur des slogans qualité.

13.3 Estimations ratées la première année

La première année a accumulé des estimations trop optimistes, surtout sur la complexité des normes aéronautiques.

Premier cas : les *ancillaries* NDC. J'avais pensé bagages et sièges comme un modèle générique simple. Aux premiers tests d'émission, la quasi-totalité des dossiers avec bébé sans siège rattaché a été rejetée. Réécriture urgente du mapping sous pression de livraison.

Sur SOAP Sabre aussi, les premières charges étaient trop basses. Le cas nominal tenait sur une slide ; le temps réel partait dans les timeouts réseau, sessions expirées et payloads inattendus.

13.4 Impact financier d'une panne

Une panne Metis touche la trésorerie des agences et la crédibilité d'Everbloo.

Un lundi à 9h30, pic d'activité : le support TourCom appelle — réservations Sabre en cascade à cause d'un certificat intermédiaire expiré sur la passerelle. Des dizaines d'agents bloqués au téléphone avec leurs clients entreprises. Si l'outil ne répond pas, le client réserve ailleurs.

Plus coûteux encore : un écart de taxes ou un segment passif en double sur Amadeus et Sabre se transforme en ADM, avec des pénalités qui peuvent monter à plusieurs milliers d'euros par dossier. D'où le temps passé sur l'arrondi bancaire, le refresh OAuth2 en tâche de fond et les contrôles pré-émission — pas du zèle, de la protection de trésorerie.

13.5 Ce que je retiens

Trois réflexes : écrire du code qu'un collègue peut reprendre, traiter les pannes externes sans improvisation, et mesurer la qualité au comportement en production plutôt qu'au nombre de frameworks.

14. Conclusion

14.1 Ce qui a tenu

Sur 38 mois, Metis a surtout montré l'écart entre modèles propres et production : sessions GDS synchrones (Sabre, Amadeus) d'un côté, API NDC sans état de l'autre. Les arbitrages se sont joués entre latence, risque financier (ADM) et parcours utilisateur.

`SabreAdapter.ts` a permis de brancher British Airways, Aegean et Turkish sans retoucher les composants panier. L'intercepteur Axios avec file d'attente a coupé les déconnexions au refresh OAuth2. `matchesPaxRefId` a ramené le rejet NDC sous 0,1 %. La modale de confirmation tarifaire a sauvé 92 % des paniers touchés par une hausse de dernière minute. Le découpage des segments passifs a évité tout ADM pour double écriture sur plus de deux ans.

14.2 Lien avec le RNCP 36137

Le titre d'**Expert en Ingénierie Informatique** (niveau 7) demande de concevoir des systèmes distribués, de tenir la qualité en production et de travailler en équipe sur des enjeux métier. Sur Metis, ça s'est joué concrètement : contrats TypeScript entre services, appels non bloquants Node.js vers Sabre/NDC, front Nuxt 3 / Vue 3 / Pinia avec une baisse de 28 % du temps de réponse moyen sur la recherche, suites Mocha / Chai / Playwright pour l'appairage passagers et l'égalité ISO 4217 sur les taxes. Les lundis matins (certificat expiré, deadlocks, écarts de centimes) et le travail quotidien avec les PO et l'équipe en Tunisie ont autant compté que les specs écrites.

14.3 Suite

Deux chantiers pour la suite de Metis. **IATA ONE Order** regroupera PNR, billet et options dans une commande unique ; le modèle pivot déjà en place limite l'impact sur les écrans existants. **OpenTelemetry** permettra de mesurer la latence par transporteur et d'isoler plus vite les ralentissements amont.

Je poursuis comme **Architecte Systèmes d'Information et Lead Developer**. Critère inchangé : un système se juge à ce qu'il fait en production pour les utilisateurs, pas à la liste des frameworks du `package.json`.

Table des sigles et abréviations

Ce tableau récapitule les principaux sigles et termes techniques de la plateforme Metis et de la distribution aérienne.

Sigle / Terme	Définition et Contexte Métier
API	<i>Application Programming Interface</i> — Interface standardisée d'échange de données.
B2B	<i>Business to Business</i> — Flux commerciaux et logiciels entre entreprises.
CI/CD	<i>Continuous Integration / Deployment</i> — Pipeline d'automatisation des tests et livraisons.
CSR	<i>Client-Side Rendering</i> — Rendu dynamique exécuté dans le navigateur du client.
DOM	<i>Document Object Model</i> — Représentation objet de la page web.
EDIFACT	<i>Electronic Data Interchange for Administration, Commerce and Transport</i> — Standard d'échange des GDS historiques.
EMD	<i>Electronic Miscellaneous Document</i> — Titre électronique pour les services auxiliaires payants.
GDS	<i>Global Distribution System</i> — Réseau mondial centralisé de distribution (Sabre, Amadeus).
IATA	<i>International Air Transport Association</i> — Organisation régissant les normes de l'aviation civile.
ISO 4217	Standard international de codification et précision monétaire des devises.
JSON	<i>JavaScript Object Notation</i> — Format textuel léger d'échange de données.
LCC	<i>Low-Cost Carrier</i> — Compagnie aérienne à bas coûts.
MVCS	<i>Model-View-Controller-Service</i> — Patron d'architecture isolant le métier et la persistance.
NDC	<i>New Distribution Capability</i> — Standard IATA XML/JSON de distribution aérienne directe.
OCN	<i>Order Change Notification</i> — Notification asynchrone de modification de commande ou vol.
ORM	<i>Object-Relational Mapping</i> — Couche d'abstraction entre base SQL et code applicatif.

Sigle / Terme	Définition et Contexte Métier (suite)
PNR	<i>Passenger Name Record</i> — Dossier passager informatique (identifiant à 6 caractères).
POS	<i>Point of Sale</i> — Point de vente définissant la localisation et les droits marchands.
REST	<i>Representational State Transfer</i> — Style d'architecture pour services web légers.
SBT	<i>Self-Booking Tool</i> — Outil de réservation en ligne pour les voyages d'affaires.
SOA	<i>Service-Oriented Architecture</i> — Architecture orientée services découplés.
SOAP	<i>Simple Object Access Protocol</i> — Protocole XML d'échanges de services web.
SSR	<i>Server-Side Rendering / Special Service Request</i> — Rendu serveur ou service spécial passager.
UI / UX	<i>User Interface / User Experience</i> — Conception et ergonomie de l'interface utilisateur.
XML	<i>Extensible Markup Language</i> — Langage de balisage structuré pour l'échange de documents.

Glossaire

Ce glossaire précise les notions de génie logiciel et de distribution aérienne utilisées dans ce mémoire.

Concept / Terme	Définition et Contexte Métier
Ancillaries	Prestations de voyage vendues en option (bagages en soute, sièges spécifiques, repas spéciaux, embarquement prioritaire).
Canonical Data Model	Patron d'architecture définissant un modèle de données pivot unique pour normaliser des flux hétérogènes.
Debit Memo (ADM)	Pénalité financière émise par une compagnie aérienne à l'encontre d'une agence pour non-respect des règles tarifaires.
Adapter Pattern	Patron de conception structurel adaptant une interface incompatible vers une interface cible unifiée.
Event Loop	Mécanisme asynchrone non-bloquant de Node.js traitant les entrées/sorties sur un thread unique.
IATA ONE Order	Standard d'aviation unifiant la commande, la billetterie et les services annexes sous un enregistrement unique.
PaxRefID	Identifiant unique normalisé d'un passager dans un message NDC pour lier voyageur, billet et services.
Pinia Store	Gestionnaire d'état réactif officiel pour Vue 3 offrant le typage strict TypeScript et la modularité.
PCC / Office ID	Identifiant marchand attribué par un GDS formalisant les droits d'émission d'un point de vente.
Reshopping	Re-tarification en temps réel avant le paiement pour valider le tarif et actualiser le panier sans rupture.
Segment Passif	Ligne de vol d'information inscrite dans un GDS pour intégration comptable sans réservation d'inventaire.
SBT	Outil d'auto-réservation en ligne dédié aux collaborateurs pour leurs voyages d'affaires en entreprise.
Sequelize ORM	Outil de mapping objet-relationnel facilitant les requêtes SQL et transactions en Node.js.

Bibliographie

Génie logiciel et architecture

- [1] **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- [2] **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman.
- [3] **Martin, R. C.** (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [4] **Newman, S.** (2021). *Building Microservices : Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
- [5] **Kleppmann, M.** (2017). *Designing Data-Intensive Applications : The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [6] **Evans, E.** (2003). *Domain-Driven Design : Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

Standards Industriels, Spécifications Aéronautiques

- [IATA-1] **International Air Transport Association (IATA).** (2021). *New Distribution Capability (NDC) Implementation Guide - Version 21.3*. IATA Publications, Montreal-Geneva.
- [IATA-2] **International Air Transport Association (IATA).** (2023). *ONE Order Standard Architecture & Technical Specifications*. IATA Future Airline Distribution, Geneva.
- [IATA-3] **Sabre Corporation.** (2024). *Sabre APIs Developer Reference : SOAP and REST Web Services Integration Manual*. Sabre Dev Studio, Southlake, Texas.
- [IATA-4] **Amadeus IT Group.** (2023). *Amadeus Web Services Integration Guide : Passive Segments & PNR Management*. Amadeus Global Documentation, Madrid.
- [IATA-5] **International Organization for Standardization.** (2015). *ISO 4217 :2015 - Codes for the representation of currencies and funds*. ISO, Geneva.

Documentations Officielles, Ressources Web

- [WEB-1] **Vue.js Core Team.** (2024). *Vue.js 3 Documentation & Composition API Guide*. Accessible en ligne : <https://vuejs.org/guide/introduction.html>.
- [WEB-2] **Nuxt Core Team.** (2024). *Nuxt 3 Framework Documentation : Server-Side Rendering and Hybrid Rendering*. Accessible en ligne : <https://nuxt.com/docs>.
- [WEB-3] **OpenJS Foundation.** (2024). *Node.js Design Patterns and Asynchronous Program-*

- ming Reference (v18/v20 LTS)*. Accessible en ligne : <https://nodejs.org/docs>.
- [WEB-4] **Sequelize Team**. (2024). *Sequelize ORM v6 API Documentation : Transactions, Migrations, and Associations*. Accessible en ligne : <https://sequelize.org>.